

WYKŁAD 9

NumPy i Matplotlib

Kurs języka Python — semestr 2025/2026

Wprowadzenie do programowania II | Informatyka Stosowana

Akademia Nauk Stosowanych w Nowym Sączu

Tematy

1	NumPy - wprowadzenie	ndarray, tworzenie tablic, typy danych
2	Indeksowanie i wycinki	slicing, fancy indexing, maski logiczne
3	Operacje na tablicach	wektoryzacja, broadcasting, agregacje
4	Benchmark: pętla vs NumPy	dlaczego NumPy jest szybsze od list
5	Zaawansowane NumPy	reshape, random, linalg, view vs copy
6	Matplotlib — podstawy	pyplot, plot(), scatter(), bar(), hist()
7	Customizacja wykresów	kolory, style, etykiety, legenda, adnotacje
8	Subplots i zapis	fig/axes, subplots(), savefig()
9	SciPy - krótki przegląd	optimize, interpolate, stats

NumPy - wprowadzenie

Numeryczne obliczenia w Pythonie

Czym jest NumPy?

Fundament ekosystemu naukowego Pythona

NumPy (Numerical Python)

- Biblioteka do obliczeń numerycznych
- Typ ndarray - tablica N-wymiarowa
- Napisana w C - szybka i wydajna
- Fundament dla Pandas, Matplotlib, SciPy
- Wektoryzacja zamiast pętli
- Broadcasting - operacje na różnych kształtach

Instalacja i import

```
pip install numpy

import numpy as np

# Wersja
print(np.__version__) # np. 2.2.3

# Pierwszy ndarray
a = np.array([1, 2, 3, 4, 5])
print(a)           # [1 2 3 4 5]
print(type(a))     # <class 'numpy.ndarray'>
```

✓ Konwencja: `import numpy as np` - używana powszechnie, nie zmieniaj aliasu na inny.

Tworzenie tablic ndarray

Różne sposoby inicjalizacji

Z listy / generatoru

```
np.array([1, 2, 3])      # z listy
np.array([[1,2],[3,4]])  # macierz 2x2

np.zeros(5)              # [0. 0. 0. 0. 0.]
np.ones((2, 3))          # macierz 2x3 jedynek
np.full((3,3), 7)        # macierz 3x3 siódemek
np.empty((2, 2))         # niezainicjalizowana!

np.eye(3)                # macierz jednostkowa 3x3
np.diag([1,2,3])         # diagonalna
```

Zakresy i wypełnianie

```
np.arange(0, 10, 2)      # [0 2 4 6 8]
np.arange(5)             # [0 1 2 3 4]

# linspace – równe odstępy
np.linspace(0, 1, 5)
# [0.  0.25  0.5  0.75  1. ]

# logspace – skala logarytmiczna
np.logspace(0, 3, 4)     # [1 10 100 1000]

# Losowe
np.random.rand(3, 3)     # uniform [0,1)
np.random.randn(3)       # rozkł. normalny
```

Atrybuty tablicy ndarray

Każda tablica ma kluczowe właściwości

```
a = np.array([[1, 2, 3],  
             [4, 5, 6]])  # macierz 2x3  
  
a.ndim      # 2          - liczba wymiarów (rank)  
a.shape     # (2, 3)     - kształt (wiersze, kolumny)  
a.size      # 6          - łączna liczba elementów  
a.dtype     # dtype('int64') - typ elementów  
a.itemsize  # 8          - bajtów na element  
a.nbytes    # 48         - całkowity rozmiar w bajtach
```

Typy danych NumPy (dtype)

<code>np.int32, np.int64</code>	Liczby całkowite (32/64 bit)
<code>np.float32, np.float64</code>	Zmiennoprzecinkowe (domyślny: float64)
<code>np.complex128</code>	Liczby zespolone
<code>np.bool_</code>	Wartości logiczne (True/False)
<code>np.str_, np.bytes_</code>	Napisy i bajty (rzadko używane)

Indeksowanie i wycinki (slicing)

Podobnie do list, ale z rozszerzeniami

Tablice 1D

```
a = np.array([10,20,30,40,50])

a[0]      # 10
a[-1]     # 50
a[1:4]    # [20 30 40]
a[::2]    # [10 30 50] co drugi
a[::-1]   # [50 40 30 20 10]

# Fancy indexing
a[[0, 2, 4]] # [10 30 50]

# Maska logiczna (boolean)
a[a > 25]    # [30 40 50]
```

Tablice 2D

```
m = np.array([[1,2,3],
              [4,5,6],
              [7,8,9]])

m[0, 1]    # 2 (wiersz 0, kol 1)
m[1]       # [4 5 6] cały wiersz
m[:, 0]    # [1 4 7] cała kolumna
m[0:2, 1:] # [[2 3]
              # [5 6]]

# Warunek na macierzy
m[m % 2 == 0] # [2 4 6 8]
```

Operacje na tablicach

Wektoryzacja, broadcasting, agregacje

Operacje elementowe (element-wise)

Operatory działają na każdym elemencie z osobna

Arytmetyka

```
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])

a + b # [11 22 33]
a * b # [10 40 90]
a ** 2 # [ 1  4  9]
b / a # [10. 10. 10.]
b // a # [10 10 10]
```

Funkcje uniwersalne (ufunc)

```
x = np.array([0, np.pi/2, np.pi])

np.sin(x) # [0.  1.  0.]
np.sqrt(a) # [1.  1.41  1.73]
np.exp(a) # [2.72  7.39 20.09]
np.log(b) # [2.30  3.00  3.40]
np.abs([-1, -2, 3]) # [1 2 3]
```

Porównania - zwracają tablicę bool

```
a = np.array([1, 2, 3, 4, 5])
a > 3 # [False False False True True]
(a > 1) & (a < 4) # [False True True False False] – & zamiast and!
np.where(a > 3, a, 0) # [0 0 0 4 5] – warunkowe podstawienie
```

Broadcasting

Automatyczne dopasowanie kształtów tablic w operacjach

Reguły broadcastingu

- 1. Porównaj wymiary od prawej do lewej
- 2. Wymiary są zgodne, jeśli równe lub jeden = 1
- 3. Wymiar = 1 jest „rozciągany” do drugiego

Przykład: skalar + tablica

```
a = np.array([1, 2, 3])  
a + 10 # [11 12 13]  
# 10 jest "rozciągnięty" do [10,10,10]  
  
a * 2 # [2 4 6]
```

Macierz + wektor

```
m = np.array([[1,2,3],  
              [4,5,6]]) # (2,3)  
v = np.array([10,20,30]) # (3,)  
  
m + v  
# [[11 22 33]  
#   [14 25 36]]  
# v dodane do KAŻDEGO wiersza
```

Kolumna + wiersz

```
col = np.array([[1],[2],[3]]) # (3,1)  
row = np.array([10, 20, 30]) # (3,)  
  
col + row  
# [[11 21 31]  
#   [12 22 32]  
#   [13 23 33]]  
# tabliczka "dodawania"!
```

Wektoryzacja - porzuc pętle!

Zamiast iterować element po elemencie, operuj na całych tablicach

❌ Podejście z pętlą (wolne)

```
data = list(range(1_000_000))

result = 0
for x in data:
    result += x ** 2

# ~150 ms na typowym CPU
```

✅ Podejście NumPy (szybkie)

```
data = np.arange(1_000_000)

result = np.sum(data ** 2)

# ~2 ms – 75x szybciej!
```

Dlaczego NumPy jest szybsze?

- Jednorodny typ danych — brak narzutu dynamicznego typowania Pythona
- Pętla wewnętrzna wykonywana w skompilowanym kodzie C/Fortran
- Ciągły blok pamięci — lepsza lokalność cache procesora
- Wykorzystanie instrukcji SIMD procesora (SSE, AVX)

Pętla vs NumPy - pomiar czasu

Suma kwadratów $n = 300\,000$ elementów

```
import time, numpy as np

n = 300_000
data_list = [i for i in range(n)]
data_np = np.arange(n, dtype=np.int64)

# Pętla Python
t0 = time.perf_counter()
result = sum(x**2 for x in data_list)
t1 = time.perf_counter()
print(f"Python: {t1-t0:.4f} s") # ~0.04 s

# NumPy
t0 = time.perf_counter()
result = np.sum(data_np ** 2)
t1 = time.perf_counter()
print(f"NumPy: {t1-t0:.6f} s") # ~0.002 s
```

Przykładowe wyniki (orientacyjne)

Metoda	Czas	Przyspieszenie
Python (pętla)	~42 ms	1x
NumPy (wektoryzacja)	~2.3 ms	~18x
Numba @jit (2. wywoł.)	~0.17 ms	~250x

Funkcje agregujące

Redukcja tablicy do jednej wartości (lub wzdłuż osi)

Podstawowe agregacje

```
a = np.array([3, 1, 4, 1, 5, 9])

np.sum(a)      # 23
np.mean(a)     # 3.833...
np.std(a)      # 2.676...
np.min(a)      # 1
np.max(a)      # 9
np.argmin(a)   # 1 (indeks min)
np.argmax(a)   # 5 (indeks max)
np.cumsum(a)   # [3 4 8 9 14 23]
```

Agregacja wzdłuż osi (axis)

```
m = np.array([[1, 2, 3],
               [4, 5, 6]])

np.sum(m)      # 21 (całość)
np.sum(m, axis=0) # [5 7 9]
# ↑ suma kolumn
np.sum(m, axis=1) # [6 15]
# ↑ suma wierszy
np.mean(m, axis=0) # [2.5 3.5 4.5]
```

⚠ axis=0 — operacja "w dół" (po wierszach, wynik: jeden wiersz)
axis=1 — operacja "w prawo" (po kolumnach, wynik: jedna kolumna)

Przekształcanie tablic

reshape, flatten, transpose

```
a = np.arange(12)          # [ 0  1  2  3  4  5  6  7  8  9 10 11]

# reshape – zmiana kształtu (bez kopiowania danych!)
b = a.reshape(3, 4)        # [[ 0  1  2  3]
                          #  [ 4  5  6  7]
                          #  [ 8  9 10 11]]

c = a.reshape(2, -1)       # -1 = automatycznie oblicz (tu: 6)
                          # [[ 0  1  2  3  4  5]
                          #  [ 6  7  8  9 10 11]]

# flatten vs ravel
b.flatten() # [0 1 2 ...11] – KOPIA (nowa tablica)
b.ravel()   # [0 1 2 ...11] – WIDOK (ten sam bufor!)

# transpose
b.T         # [[ 0  4  8]
            #  [ 1  5  9]
            #  [ 2  6 10]
            #  [ 3  7 11]] – transponowana (4x3)

# łączenie
np.concatenate([a[:6], a[6:]]) # [0..11]
np.vstack([b[0], b[1]])        # 2 wiersze
np.hstack([b[:, :2], b[:, 2:]]) # złącz kolumny
```

Generowanie losowe - np.random

Nowy styl (Generator) vs stary styl (legacy)

Nowy styl (zalecany od NumPy 1.17+)

```
rng = np.random.default_rng(42)

rng.random(5)          # 5 z [0, 1)
rng.integers(0, 10, size=5) # 5 int z [0, 10)
rng.normal(0, 1, (3,3)) # macierz 3x3
rng.choice([1,2,3], size=10)
rng.shuffle(arr)        # tasowanie
```

Stary styl (legacy, wciąż działa)

```
np.random.seed(42)      # ziarno

np.random.rand(3, 3)    # uniform
np.random.randn(5)      # normalny
np.random.randint(0, 10, 5)
np.random.choice(a, 3)
np.random.shuffle(arr)

# Uwaga: globalny stan –
# niezalecany w kodzie produkcyjnym
```

✅ Nowy styl `default_rng()` daje lepszą losowość, izolację stanu i jest zalecany w nowym kodzie.

Algebra liniowa - np.linalg

Podstawowe operacje macierzowe

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Mnożenie macierzowe (≠ element-wise!)
A @ B          # [[19 22] [43 50]]
np.dot(A, B)   # to samo
A * B          # [[ 5 12] [21 32]] ← element-wise!

# Właściwości macierzy
np.linalg.det(A)      # -2.0 (wyznacznik)
np.linalg.inv(A)      # macierz odwrotna
np.linalg.eig(A)      # wartości i wektory własne
np.linalg.norm(A)     # norma Frobeniusa

# Układ równań Ax = b
b = np.array([5, 11])
x = np.linalg.solve(A, b) # x = [1. 2.]

# Transpozycja
A.T              # [[1 3] [2 4]]
```


Widok (view) vs kopia (copy)

Najczęstsza pułapka NumPy - modyfikacja „oryginału” przez widok

❌ Widok - wspólny bufor pamięci

```
a = np.array([1, 2, 3, 4, 5])
b = a[1:4]      # b to WIDOK na a!

b[0] = 99
print(a)        # [1 99 3 4 5] ← !!
# Zmiana b zmieniła też a!

# Czy to widok?
b.base is a     # True
```

✅ Kopia - oddzielny bufor

```
a = np.array([1, 2, 3, 4, 5])
b = a[1:4].copy() # KOPIA!

b[0] = 99
print(a) # [1 2 3 4 5] ← OK
# a pozostało bez zmian

# Fancy indexing też tworzy kopię
c = a[[0, 2, 4]] # kopia
```

Widok (view)

slicing `a[1:4]`, `reshape()`, `ravel()`, `transpose/T`

Kopia (copy)

fancy indexing `a[[0,2]]`, `.copy()`, `.flatten()`

Matplotlib

Wizualizacja danych w Pythonie

Czym jest Matplotlib?

Najbardziej popularna biblioteka do wykresów w Pythonie

Cechy

- Inspirowana MATLABem
- Dwa interfejsy: pyplot (prosty) i OO (elastyczny)
- Eksport: PNG, PDF, SVG, EPS...
- Integracja z NumPy, Pandas, Jupyter
- Alternatywy: seaborn, plotly, bokeh

Instalacja i import

```
pip install matplotlib

import matplotlib.pyplot as plt
import numpy as np

# Najprostszy wykres
x = np.linspace(0, 2*np.pi, 100)
y = np.sin(x)
plt.plot(x, y)
plt.show()
```

Hierarchia obiektów Matplotlib



Wykres liniowy - plt.plot()

Najczęściej używany typ wykresu

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 2 * np.pi, 200)

plt.figure(figsize=(8, 4))          # rozmiar w calach

plt.plot(x, np.sin(x), label='sin(x)', color='blue', linewidth=2)
plt.plot(x, np.cos(x), label='cos(x)', color='red', linestyle='--')

plt.title('Funkcje trygonometryczne')
plt.xlabel('x [rad]')
plt.ylabel('y')
plt.legend()                        # legenda na podstawie label
plt.grid(True, alpha=0.3)          # siatka
plt.xlim(0, 2 * np.pi)
plt.ylim(-1.5, 1.5)

plt.tight_layout()                 # dopasuj marginesy
plt.savefig('trig.png', dpi=150)   # zapis do pliku
plt.show()                         # wyświetl
```

Customizacja - kolory, style, markery

Parametry plot() i gotowe style

Parametry linii

```
# Kolory: 'r','g','b','k','#FF5733'
# Styl: '-', '--', '-.', ':'
# Marker: 'o','s','^','D','*','.'
# Skrót formatowania:
plt.plot(x, y, 'r--o') # red,dash,circle
plt.plot(x, y, 'g:^') # green,dot,triangle
```

Gotowe style

```
# Dostępne style:
print(plt.style.available)
# ['seaborn-v0_8', 'ggplot', 'dark_background'
#  'fivethirtyeight', 'bmh', ...]

plt.style.use('seaborn-v0_8')
# lub tymczasowo:
with plt.style.context('ggplot'):
    plt.plot(x, y)
```

Dodatkowe opcje formatowania

```
plt.plot(x, y, color='#306998', linewidth=2.5, linestyle='-',
         marker='o', markersize=4, markerfacecolor='white',
         markeredgecolor='#306998', alpha=0.8, zorder=3)

plt.fill_between(x, y1, y2, alpha=0.2) # wypełnienie między krzywymi
plt.axhline(y=0, color='gray', ls='--') # linia pozioma
plt.axvline(x=np.pi, color='red') # linia pionowa
plt.text(np.pi, 0.1, r'$\pi$', fontsize=14) # tekst + LaTeX
```

Wykres punktowy - plt.scatter()

Podstawowy scatter

```
rng = np.random.default_rng(42)
x = rng.normal(0, 1, 100)
y = 2*x + rng.normal(0, 0.5, 100)

plt.figure(figsize=(6, 4))
plt.scatter(x, y,
            c='steelblue',      # kolor
            s=40,               # rozmiar punktów
            alpha=0.6,          # przezroczystość
            edgecolors='white', # obwódka
            linewidth=0.5)

plt.xlabel('Zmienna X')
plt.ylabel('Zmienna Y')
plt.title('Korelacja')
plt.tight_layout()
plt.show()
```

Zaawansowany scatter

```
# Kolor i rozmiar zależne od danych
colors = rng.random(100)
sizes = rng.integers(20, 200, 100)

plt.scatter(x, y,
            c=colors,          # colormap
            s=sizes,           # rozmiar
            cmap='viridis',    # paleta kolorów
            alpha=0.7)

plt.colorbar(label='Wartość')

# Dostępne palety:
# 'viridis', 'plasma', 'inferno',
# 'magma', 'cividis', 'coolwarm',
# 'RdYlGn', 'Spectral'
```

Słupkowy i histogram - bar() / hist()

Wykres słupkowy

```
langs = ['Python', 'Java', 'C++', 'JS']
users = [31, 17, 12, 25]
colors = ['#306998', '#f89820',
          '#00599C', '#f7df1e']

plt.bar(langs, users, color=colors)
plt.ylabel('Popularność [%]')
plt.title('Języki programowania')

# Poziomy:
plt.barh(langs, users, color=colors)
```

Histogram

```
data = rng.normal(170, 10, 1000)

plt.hist(data, bins=30,
         color='steelblue',
         edgecolor='white',
         alpha=0.7)
plt.xlabel('Wzrost [cm]')
plt.ylabel('Liczba osób')
plt.title('Rozkład wzrostu')

# Normalizacja (gęstość):
plt.hist(data, bins=30, density=True)
```

Inne typy wykresów:

plt.pie() - kołowy | plt.boxplot() - pudełkowy | plt.errorbar() - z błędami |
plt.contour() - warstwowy | plt.imshow() - obrazy

Dwa interfejsy: pyplot vs OO

pyplot - szybki skrypt; obiektowy (OO) - pełna kontrola

Interfejs pyplot (stanowy)

```
# Prosty, MATLAB-owy styl
# Operacje na "bieżącym" wykresie
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 4))
plt.plot(x, y, 'b-', label='dane')
plt.title('Prosty wykres')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.show()

# ✅ Szybki do prototypowania
# ❌ Niejasne "co jest aktywne"
```

Interfejs obiektowy (zalecany)

```
# Jawne referencje do fig i axes
fig, ax = plt.subplots(figsize=(8,4))

ax.plot(x, y, 'b-', label='dane')
ax.set_title('Prosty wykres')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.legend()
ax.grid(True)

fig.tight_layout()
fig.savefig('wykres.png', dpi=150)
plt.show()

# ✅ Jasne, który wykres edytujemy
# ✅ Wymagany przy subplots
```


Wiele wykresów - subplots()

Siatka wykresów na jednym rysunku

Siatka 2x2

```
fig, axes = plt.subplots(2, 2,
    figsize=(10, 8))

axes[0, 0].plot(x, np.sin(x))
axes[0, 0].set_title('sin(x)')

axes[0, 1].plot(x, np.cos(x), 'r')
axes[0, 1].set_title('cos(x)')

axes[1, 0].plot(x, np.tan(x), 'g')
axes[1, 0].set_ylim(-5, 5)
axes[1, 0].set_title('tan(x)')

axes[1, 1].plot(x, np.exp(-x), 'm')
axes[1, 1].set_title('exp(-x)')

fig.suptitle('Przegląd funkcji')
fig.tight_layout()
plt.show()
```

Inne układy

```
# 1 wiersz, 3 kolumny
fig, (ax1, ax2, ax3) = \
    plt.subplots(1, 3, figsize=(12,4))

# Wspólna oś Y
fig, axes = plt.subplots(1, 2,
    sharey=True, figsize=(10, 4))

# Różne rozmiary – subplot2grid
fig = plt.figure(figsize=(10, 6))
ax1 = plt.subplot2grid((2,3),
    (0,0), colspan=2) # duży
ax2 = plt.subplot2grid((2,3),
    (0,2))           # mały
ax3 = plt.subplot2grid((2,3),
    (1,0), colspan=3) # cały dół

# Lub: fig.add_gridspec(2, 3)
```

Adnotacje i tekst na wykresie

```
fig, ax = plt.subplots(figsize=(8, 5))
x = np.linspace(0, 2*np.pi, 200)
ax.plot(x, np.sin(x), 'b-', lw=2)

# Tekst w dowolnym miejscu
ax.text(1, 0.8, 'Maksimum', fontsize=12, color='red',
        ha='center') # ha = horizontal alignment

# Adnotacja ze strzałką
ax.annotate('Minimum', xy=(3*np.pi/2, -1), # punkt docelowy
            xytext=(4.5, -0.5), # tekst
            fontsize=11, color='green',
            arrowprops=dict(arrowstyle='->', color='green'))

# Wzór LaTeX
ax.set_title(r'Funkcja  $y = \sin(x)$ ', fontsize=16)
ax.text(5, 0.5, r' $\int_0^{2\pi} \sin(x) dx = 0$ ',
        fontsize=14, bbox=dict(boxstyle='round', facecolor='wheat'))

# Legenda z własną pozycją
ax.legend(['sin(x)'], loc='upper right')
# loc: 'best', 'upper left', 'lower right', 'center' ...
```

Zapisywanie wykresów - savefig()

Parametry savefig()

```
fig.savefig('wykres.png', dpi=300)
fig.savefig('wykres.pdf')
fig.savefig('wykres.svg')

# Opcje:
fig.savefig('w.png',
            dpi=300,          # rozdzielczość
            bbox_inches='tight', # przytnij
            facecolor='white',  # tło
            transparent=True)   # przeźr. tło
```

Formaty wyjściowe

PNG	Rastrowy, domyślny, świetny do ekranu
PDF	Wektorowy, idealny do publikacji
SVG	Wektorowy, do stron www
EPS	Wektorowy, LaTeX
JPG	Rastrowy, do zdjęć (nie wykresów)

⚠ savefig() MUSI być przed show()! Po show() Figure jest „wyczyszczone”.
Użyj fig.savefig() (OO) zamiast plt.savefig() dla pewności.

Typowe pułapki Matplotlib

✗ `plt.show()` blokuje program

```
plt.plot(x, y)
plt.show() # blokuje do zamknięcia!
# kod poniżej się nie wykona

# Rozwiązanie: plt.ion() (interactive)
# lub: plt.show(block=False)
# lub: użyj Jupyter Notebook
```

✗ Zapomniany `plt.close()`

```
# Wyciek pamięci w pętli!
for i in range(100):
    fig, ax = plt.subplots()
    ax.plot(data[i])
    fig.savefig(f'plot_{i}.png')
    plt.close(fig) # <-- KRYTYCZNE!

# plt.close('all') zamyka wszystkie
```

✗ Nachodzenie elementów

```
# Tytuł nachodzi na subplot
fig.suptitle('Tytuł główny')

# Rozwiązanie:
fig.tight_layout()
# lub: fig.subplots_adjust(top=0.9)
```

✗ Krzywizna osi

```
# Proporcje zniekształcone
plt.scatter(x, y) # kwadrat?
plt.axis('equal') # równe skale
plt.gca().set_aspect('equal')

# Odwrotna oś Y (np. dla obrazów)
plt.gca().invert_yaxis()
```

Praktyczne przykłady

NumPy + Matplotlib w działaniu

Rysowanie wielomianu

NumPy do obliczeń, Matplotlib do wizualizacji

```
import numpy as np
import matplotlib.pyplot as plt

# Wejście: "5*x**4 + 2*x**3 - x + 6, -10 10"
wejście = input("Podaj wielomian i przedział: ")
formula, zakres = wejście.rsplit(',', 1)
x_min, x_max = map(float, zakres.split())

x_val = np.linspace(x_min, x_max, 200)

# UWAGA: eval() jest potencjalnie niebezpieczny!
x = x_val # zmienna używana przez eval
y_val = eval(formula)

fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(x_val, y_val, 'b-', linewidth=2)
ax.set_xlabel('x')
ax.set_ylabel('f(x)')
ax.set_title(f'Wykres: f(x) = {formula.strip()}')
ax.grid(True, alpha=0.3)
fig.tight_layout()
plt.show()
```

Analiza statystyczna z wizualizacją

```
rng = np.random.default_rng(42)
dane = rng.normal(loc=170, scale=10, size=500) # wzrost [cm]

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Histogram z linią średniej
axes[0].hist(dane, bins=25, color='steelblue', edgecolor='white', alpha=0.7)
axes[0].axvline(np.mean(dane), color='red', ls='--', lw=2,
               label=f'średnia = {np.mean(dane):.1f} cm')
axes[0].axvline(np.median(dane), color='green', ls=':', lw=2,
               label=f'mediana = {np.median(dane):.1f} cm')
axes[0].legend()
axes[0].set_xlabel('Wzrost [cm]')
axes[0].set_title('Rozkład wzrostu')

# Boxplot
axes[1].boxplot(dane, vert=True, patch_artist=True,
               boxprops=dict(facecolor='lightblue'))
axes[1].set_ylabel('Wzrost [cm]')
axes[1].set_title(f'std = {np.std(dane):.1f}, n = {len(dane)}')

fig.suptitle('Analiza danych - NumPy + Matplotlib', fontsize=14)
fig.tight_layout()
plt.show()
```

Animacja - FuncAnimation

Dynamiczne wykresy (np. wizualizacja sortowania)

```
from matplotlib.animation import FuncAnimation
import matplotlib.pyplot as plt
import numpy as np

fig, ax = plt.subplots(figsize=(8, 4))
x = np.linspace(0, 2*np.pi, 200)
line, = ax.plot(x, np.sin(x))      # początkowa krzywa
ax.set_ylim(-1.5, 1.5)

def update(frame):
    """Aktualizacja każdej klatki."""
    line.set_ydata(np.sin(x + frame / 10))
    return line,

ani = FuncAnimation(
    fig,                # rysunek
    update,             # funkcja aktualizująca
    frames=200,         # liczba klatek
    interval=30,        # ms między klatkami
    blit=True           # optymalizacja rysowania
)
# ani.save('anim.gif', writer='pillow', fps=30)
plt.show()
```


Mapy ciepła i obrazy - imshow()

Mapa ciepła (heatmap)

```
data = rng.random((10, 10))

fig, ax = plt.subplots()
im = ax.imshow(data,
               cmap='viridis',
               interpolation='nearest')
fig.colorbar(im)
ax.set_title('Mapa ciepła')

# Macierz korelacji
corr = np.corrcoef(
    rng.random((5, 100)))
ax.imshow(corr, cmap='coolwarm',
          vmin=-1, vmax=1)
```

Wyświetlanie obrazów

```
# Obraz to tablica NumPy!
# RGB: (height, width, 3)
# Grayscale: (height, width)

img = plt.imread('photo.png')
print(img.shape) # (480,640,3)
print(img.dtype) # float32

fig, axes = plt.subplots(1, 3)
axes[0].imshow(img)
axes[1].imshow(img[:, :, 0],
               cmap='Reds') # kanał R
axes[2].imshow(
    np.mean(img, axis=2),
    cmap='gray') # grayscale

for ax in axes:
    ax.axis('off') # ukryj osie
```

SciPy - krótki przegląd

Rozszerzenie NumPy o zaawansowane algorytmy naukowe

Moduły SciPy

- `scipy.optimize` — minimalizacja, szukanie zer
- `scipy.interpolate` — interpolacja
- `scipy.stats` — rozkłady, testy statystyczne
- `scipy.linalg` — rozszerzona algebra liniowa
- `scipy.signal` — przetwarzanie sygnałów
- `scipy.fft` — szybka transformata Fouriera

Przykład: dopasowanie krzywej

```
from scipy.optimize import curve_fit

def model(x, a, b):
    return a * np.exp(b * x)

x = np.linspace(0, 4, 50)
y = 2.5 * np.exp(0.7*x) + \
    rng.normal(0, 1, 50)

popt, pcov = curve_fit(model, x, y)
# popt = [a, b] — dopasowane param.
```

Przykład: test statystyczny

```
from scipy import stats

a = rng.normal(100, 15, 50)
b = rng.normal(110, 15, 50)

stat, pval = stats.ttest_ind(a, b)
print(f"p-value = {pval:.4f}")
```

Przykład: interpolacja

```
from scipy.interpolate import interp1d

xp = np.array([0, 1, 2, 3, 4])
yp = np.array([0, 1, 4, 9, 16])

f = interp1d(xp, yp, kind='cubic')
x_new = np.linspace(0, 4, 100)
y_new = f(x_new)
```

Numba - kompilacja JIT dla Pythona

Jeszcze szybciej niż NumPy? @jit i @vectorize

@jit — kompilacja do kodu maszynowego

```
from numba import jit
import numpy as np

@jit(nopython=True)
def sum_sq(data):
    result = 0
    for x in data:
        result += x ** 2
    return result

data = np.arange(300_000, dtype=np.int64)
sum_sq(data) # 1. wywoł.: kompilacja
sum_sq(data) # 2. wywoł.: ~0.17 ms!
```

Warianty dekoratora @jit

@jit(nopython=True)	Maksymalna wydajność
@njit	Alias dla nopython=True
@jit(fastmath=True)	Szybsza mat. (mniej dokładna)
@jit(parallel=True)	Równoległe wątki + prange()
@vectorize	Wektoryzacja z typami

⚠ Numba kompiluje przy PIERWSZYM wywołaniu zysk widoczny dopiero od drugiego.
Nie każdy kod da się skompilować (ograniczenia nopython mode).

Przydatne triki NumPy

```
# Sortowanie
a = np.array([3, 1, 4, 1, 5])
np.sort(a)          # [1 1 3 4 5]
np.argsort(a)       # [1 3 0 2 4]

# Unikalne wartości
np.unique([1,1,2,3,3]) # [1 2 3]
np.unique(a, return_counts=True)
# (array([1,3,4,5]), array([2,1,1,1]))

# Zapisywanie / wczytywanie
np.save('dane.npy', a)
b = np.load('dane.npy')

# Wiele tablic
np.savez('dane.npz', x=a, y=b)
d = np.load('dane.npz')
d['x'], d['y']
```

```
# Meshgrid (do wykresów 3D)
x = np.linspace(-2, 2, 100)
y = np.linspace(-2, 2, 100)
X, Y = np.meshgrid(x, y)
Z = np.sin(X**2 + Y**2)

# np.where – warunkowe wybieranie
arr = np.array([-2, -1, 0, 1, 2])
np.where(arr > 0, arr, 0)
# [0 0 0 1 2]

# np.clip – ograniczenie zakresu
np.clip(arr, -1, 1)
# [-1 -1  0  1  1]

# np.nan* – ignorowanie NaN
a = np.array([1, np.nan, 3])
np.nanmean(a)    # 2.0
np.nansum(a)     # 4.0
```

NumPy i Matplotlib

1	ndarray to fundament NumPy	jednorodny typ, ciągły blok pamięci, N-wymiary
2	Wektoryzacja zamiast pętli	operacje element-wise, broadcasting, ufunc
3	NumPy jest 10–100x szybsze od list	kod C, SIMD, cache-friendly layout
4	view vs copy — kluczowa pułapka	slicing tworzy widok, .copy() tworzy kopię
5	Matplotlib: pyplot vs OO	pyplot do prototypów, OO do produkcji
6	subplots() + tight_layout()	wiele wykresów, automatyczne marginesy
7	savefig() PRZED show()	eksport PNG/PDF/SVG z odpowiednim dpi
8	SciPy rozszerza NumPy	optymalizacja, statystyka, interpolacja, sygnały